UnoSolver.jl, a unified SQP and barrier solver for nonlinearly constrained optimization

Alexis Montoison [1] **and Charlie Vanaret** [2]

1 Mathematics and Computer Science Division, Argonne National Laboratory, USA **2** Applied Optimization Department, Zuse-Institut Berlin, Germany

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor: Open Journals

Reviewers:

- @openjournals

Submitted: 01 January 1970
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

DRAFT

Summary

UnoSolver.jl is the Julia interface to Uno, a modular C++ solver for nonlinearly constrained optimization. Uno unifies Lagrange-Newton methods such as SQP and barrier methods by combining building blocks such as strategies to compute descent directions, problem reformulations, and globalization techniques.

Statement of need

Gala is an Astropy-affiliated Python package for galactic dynamics. Python enables wrapping low-level languages (e.g., C) for speed without losing flexibility or ease-of-use in the user-interface. The API for Gala was designed to provide a class-based and user-friendly interface to fast (C or Cython-optimized) implementations of common operations such as gravitational potential and force evaluation, orbit integration, dynamical transformations, and chaos indicators for nonlinear dynamics. Gala also relies heavily on and interfaces well with the implementations of physical units and astronomical coordinate systems in the Astropy package (Astropy Collaboration, 2013) (astropy.units and astropy.coordinates).

Gala was designed to be used by both astronomical researchers and by students in courses on gravitational dynamics or astronomy. It has already been used in a number of scientific publications (Pearson et al., 2017) and has also been used in graduate courses on Galactic dynamics to, e.g., provide interactive visualizations of textbook material (Binney & Tremaine, 2008). The combination of speed, design, and support for Astropy functionality in Gala will enable exciting scientific explorations of forthcoming data releases from the *Gaia* mission (Gaia Collaboration, 2016) by students and experts alike.

State of the field

Several tools exist for galactic dynamics computations:
galpy (**?**) is a Python package with similar goals, providing orbit integration and potential classes for galactic dynamics.
NEMO (**?**) is a well-established, comprehensive stellar dynamics toolbox written primarily in C, offering extensive functionality but with a steeper learning curve and less integration with modern Python workflows.
Other tools like GalPot provide specific Milky Way potential models but lack the broader dynamical analysis capabilities.

Gala was built rather than contributing to existing projects for several reasons. First, Gala was designed from the ground up to integrate seamlessly with the Astropy ecosystem, using astropy.units and astropy.coordinates

as core dependencies rather than optional features. This tight integration enables natural workflows for astronomers already using Astropy. Second, `Gala`'s object-oriented API with consistent interfaces across subpackages (potentials, integrators, dynamics) provides a more modular and extensible design than alternatives available at the time. Third, `Gala` fills a specific niche between simple demonstration codes and full N-body simulation packages like `Gadget` (**?**) – it focuses on the common tasks in galactic dynamics research (orbit integration, potential evaluation, coordinate transformations) while maintaining both performance through C implementations and usability through its Python interface.

Software design

`Gala`'s design philosophy is based on three core principles: (1) to provide a user-friendly, modular, object-oriented API, (2) to use community tools and standards (e.g., Astropy for coordinates and units handling), and (3) to use low-level code (C/C++/Cython) for performance while keeping the user interface in Python. Within each of the main subpackages in `gala` (`gala.potential`, `gala.dynamics`, `gala.integrate`, etc.), we try to maintain a consistent API for classes and functions. For example, all potential classes share a common base class and implement methods for computing the potential, forces, density, and other derived quantities at given positions. This also works for compositions of potentials (i.e., multi-component potential models), which share the potential base class but also act as a dictionary-like container for different potential components. As another example, all integrators implement a common interface for numerically integrating orbits. The integrators and core potential functions are all implemented in C without support for units, but the Python layer handles unit conversions and prepares data to dispatch to the C layer appropriately.Within the coordinates subpackage, we extend Astropy's coordinate classes to add more specialized coordinate frames and transformations that are relevant for Galactic dynamics and Milky Way research.

Research impact statement

`Gala` has demonstrated significant research impact and grown both its user base and contributor community since its initial release. The package has evolved through contributions from over 18 developers beyond the original core developer ((**?**)), with community members adding new features, reporting bugs, and suggesting new features.

While `Gala` started as a tool primarily to support the core developer's research, it has expanded organically to support a range of applications across domains in astrophysics related to Milky Way and galactic dynamics. The package has been used in over 400 publications (according to Google Scholar) spanning topics in galactic dynamics such as modeling stellar streams (Pearson et al., 2017), Milky Way mass modeling, and interpreting kinematic and stellar population trends in the Galaxy. `Gala` is integrated within the Astropy ecosystem as an affiliated package and has built functionality that extends the widely-used `astropy.units` and `astropy.coordinates` subpackages. `Gala`'s impact extends beyond citations in research: Because of its focus on usability and user interface design, `Gala` has also been incorporated into graduate-level galactic dynamics curricula at multiple institutions.

`Gala` has been downloaded over 100,000 times from PyPI and conda-forge yearly (or ~2,000 downloads per week) over the past few years, demonstrating a broad and active user community. Users span career stages from graduate students to faculty and other established researchers and represent institutions around the world. This broad adoption and active participation validate `Gala`'s role as core community infrastructure for galactic dynamics research.

DRAFT

Mathematics

Single dollars ($) are required for inline mathematics e.g. $f(x) = e^{\pi/x}$

Double dollars make self-standing equations:

$$\Theta(x) = \left\{ \begin{array}{l} 0 \text{ if } x < 0 \\ 1 \text{ else} \end{array} \right.$$

You can also use plain LaTeXfor equations

$$\hat{f}(\omega) = \int_{-\infty}^{\infty} f(x) e^{i\omega x} dx \tag{1}$$

and refer to Equation 1 from text.

Citations

Citations to entries in paper.bib should be in rMarkdown format.

If you want to cite a software repository URL (e.g. something on GitHub without a preferred citation) then you can do it with the example BibTeX entry below for Smith et al. (2020).

For a quick reference, the following citation commands can be used: - `@author:2001` -> "Author et al. (2001)" - `[@author:2001]` -> "(Author et al., 2001)" - `[@author1:2001; @author2:2001]` -> "(Author1 et al., 2001; Author2 et al., 2002)"

Figures

Figures can be included like this: Caption for example figure. and referenced from text using section 8.

Figure sizes can be customized by adding an optional second parameter: Caption for example figure.

AI usage disclosure

No generative AI tools were used in the development of this software, the writing of this manuscript, or the preparation of supporting materials.

Acknowledgements

We acknowledge contributions from Brigitta Sipocz, Syrtis Major, and Semyeong Oh, and support from Kathryn Johnston during the genesis of this project.

References

Astropy Collaboration. (2013). Astropy: A community Python package for astronomy. *Astronomy and Astrophysics*, *558*. https://doi.org/10.1051/0004-6361/201322068

Binney, J., & Tremaine, S. (2008). *Galactic Dynamics: Second Edition*. Princeton University Press. http://adsabs.harvard.edu/abs/2008gady.book.....B

DRAFT

Gaia Collaboration. (2016). The Gaia mission. *Astronomy and Astrophysics*, *595*. https://doi.org/10.1051/0004-6361/201629272

Pearson, S., Price-Whelan, A. M., & Johnston, K. V. (2017). Gaps in Globular Cluster Streams: Pal 5 and the Galactic Bar. *ArXiv e-Prints*. http://adsabs.harvard.edu/abs/2017arXiv170304627P

Smith, A. M., Thaney, K., & Hahnel, M. (2020). Fidgit: An ungodly union of GitHub and figshare. In *GitHub repository*. GitHub. https://github.com/arfon/fidgit

DRAFT